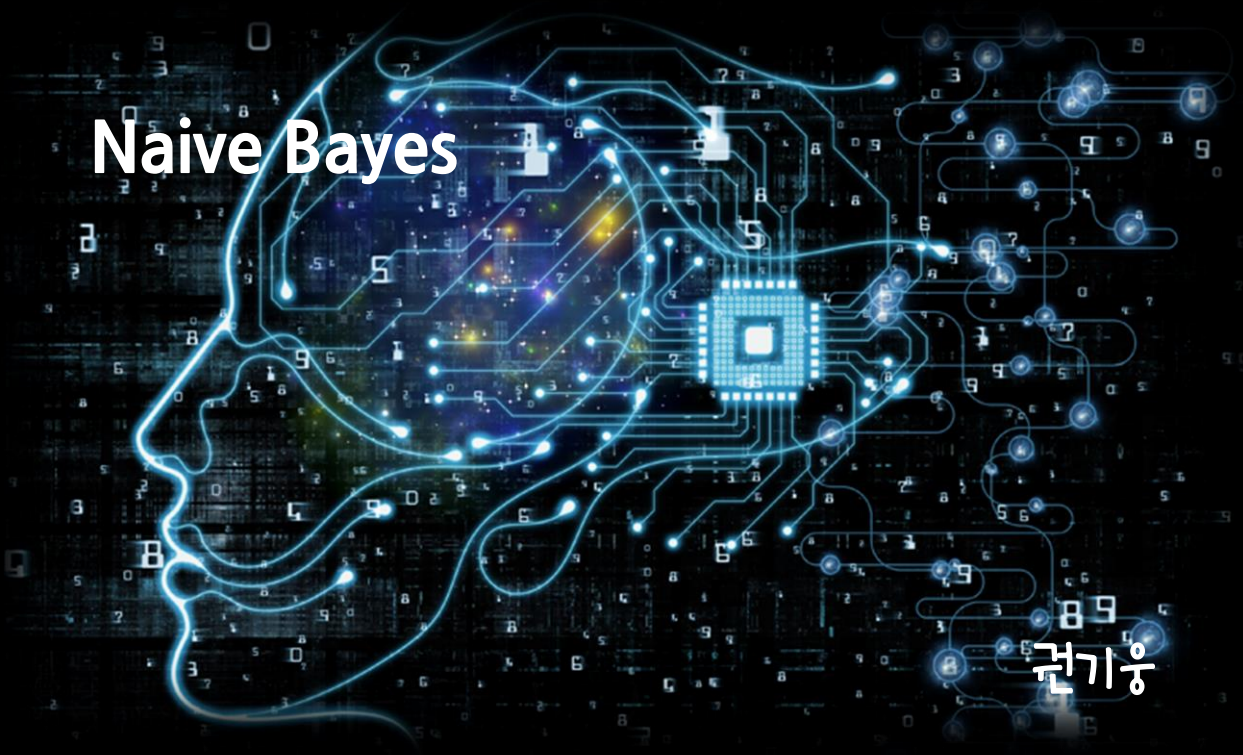
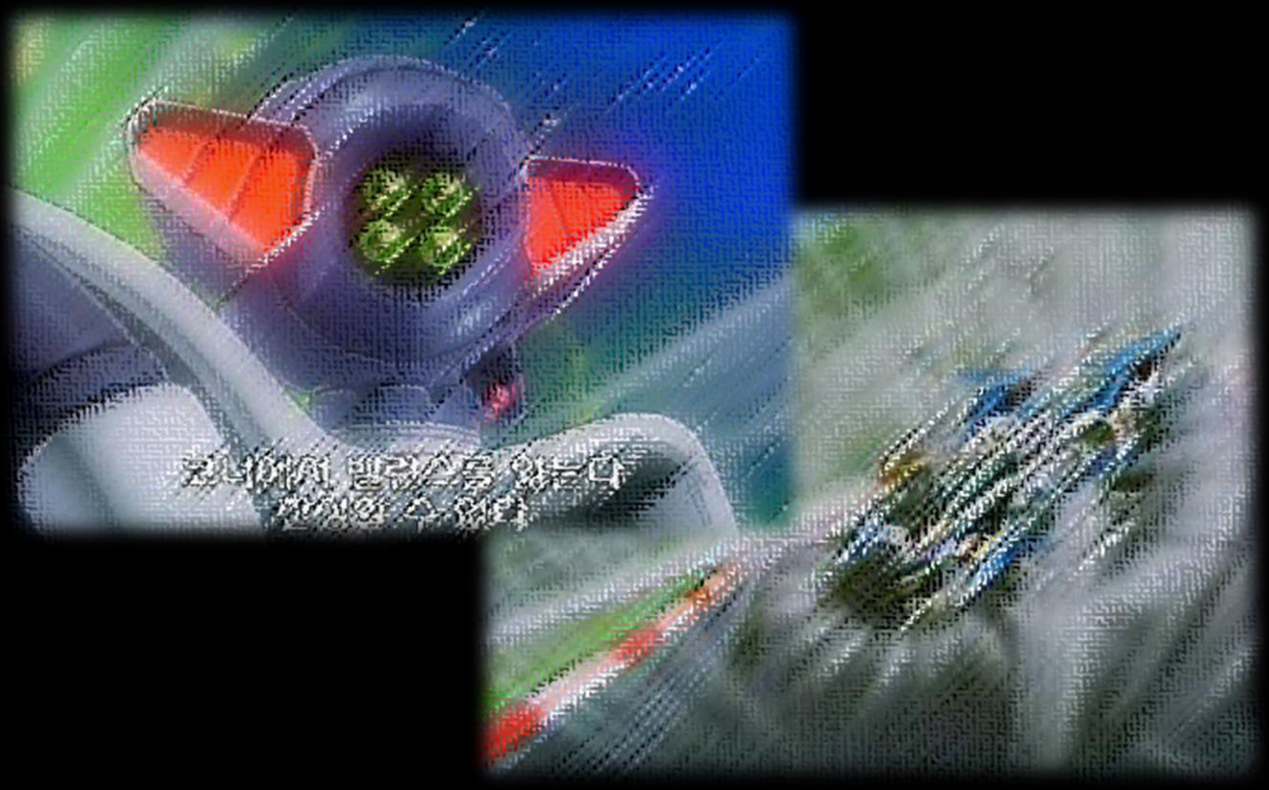


Naive Bayes

권기웅



1. Naive Bayes

1.1 Naive Bayes

1.1.1 Naive Bayes

문장		
구분	문장	욕1아니면0
A	you are a dog	1
B	maybe not take him to dog park stupid	1
C	my dalmation is so cute I love him	0
D	I am a boy	0
Z	You are a stupid dog	?

- 문장 A 는 욕, 문장 B 는 욕 , ... 을 학습시키고
- 문장 Z 가 욕인지 아닌지 구하는 문제
- Z 가 욕일 확률, 욕이 아닐 확률을 구해서
- 확률이 더 높은 쪽을 답으로

Z는 욕일 확률	0.50
Z는 욕이 아닐 확률	0.08
Z는 욕	

2. 욕 필터링

2.1 NaiveBayes.java

2.1.1 단어들 중복제거 하는 메소드

```
// 단어들 중복제거해서 String[]로
private static String[] getWordSet(ArrayList<String[]> datas) {
    HashSet<String> wordList = new HashSet<>();
    for (String[] words : datas) {
        for (String w : words) {
            wordList.add(w.toLowerCase());
        }
    }
    String[] wordSet = wordList.toArray(new String[wordList.size()]);
    Arrays.sort(wordSet);
    return wordSet;
}
```

2.1.2 단어 나오는지 체크하는 메소드

```
// 문장속에 단어가 나오면 1, 안나오면 0
// p : String[] wordSet, String[] sentence
private static ArrayList<Integer> getResultSet(
        String[] wordSet,   // [a, are, boy, you]
        String[] sentence) {// [i, am, a, boy]
    ArrayList<Integer> result = new ArrayList<>();
    boolean contains;
    for (String w : wordSet) {
        contains = false;

        for (String s : sentence) {
            if (w.equals(s.toLowerCase())) {
                contains = true;
                break;
            }
        }

        if (contains) {
            result.add(1);
        } else {
            result.add(0);
        }
    }
    return result; // [1, 0, 1, 0]
}
```

2.1.3 확률 계산하는 메소드

```java
// 확률 계산
// p : ArrayList<String[]> datas, ArrayList<Integer> results
public static HashMap<String, Object> train(ArrayList<String[]> datas, ArrayList<
    HashMap<String, Object> trainData = new HashMap<>();
    ArrayList<Double> pGood = new ArrayList<>();
    ArrayList<Double> pBad = new ArrayList<>();

    String[] wordSet = NaiveBayes.getWordSet(datas);

    ArrayList<ArrayList<Integer>> trainMatrix = new ArrayList<>();
    for (String[] sentence : datas) {
        trainMatrix.add(NaiveBayes.getResultSet(wordSet, sentence));
    }

    int numTrainSentences = trainMatrix.size();
    int numWords = trainMatrix.get(0).size();

    ArrayList<Integer> goodCount = new ArrayList<>();
    for (int i = 0; i < numWords; i++) {
        goodCount.add(0);
    }

    ArrayList<Integer> badCount = new ArrayList<>();
    for (int i = 0; i < numWords; i++) {
        badCount.add(0);
    }
```

```java
    double goodAllCount = 0;
    double badAllCount = 0;
    for (int i = 0; i < numTrainSentences; i++) {
        if (results.get(i) == 1) { // i번째 문장이 안좋음
            for (int j = 0; j < badCount.size(); j++) {
                badCount.set(j, badCount.get(j) + trainMatrix.get(i).get(j));
            }
            for (Integer t : trainMatrix.get(i)) {
                badAllCount += t;
            }
        } else { // i번째 문장이 좋음
            for (int j = 0; j < goodCount.size(); j++) {
                goodCount.set(j, goodCount.get(j) + trainMatrix.get(i).get(j));
            }
            for (Integer t : trainMatrix.get(i)) {
                goodAllCount += t;
            }
        }
    }

    for (Integer i : badCount) {pBad.add(i / badAllCount);}
    for (Integer i : goodCount) {pGood.add(i / goodAllCount);}

    trainData.put("wordSet", wordSet);
    trainData.put("results", results);
    trainData.put("pGood", pGood);
    trainData.put("pBad", pBad);
    return trainData;
}
```

2.1.4 결과 구하는 메소드

```java
// 결과 구하기
// p : String testData, HashMap<String, Object> trainData
// r : 나쁘면 1, 좋으면 0
@SuppressWarnings("unchecked")
public static int getResult(String testData, HashMap<String, Object> trainData) {
    String[] testEntry = testData.split(" ");
    String[] wordSet = (String[]) trainData.get("wordSet");
    ArrayList<Integer> testResultSet = NaiveBayes.getResultSet(wordSet, testEntry

    ArrayList<Double> pGood = (ArrayList<Double>) trainData.get("pGood");
    ArrayList<Double> pBad = (ArrayList<Double>) trainData.get("pBad");

    double pG = 0;
    double pB = 0;

    for (int i = 0; i < testResultSet.size(); i++) {
        if (testResultSet.get(i) == 1) {
            pB += testResultSet.get(i) * pBad.get(i);
            pG += testResultSet.get(i) * pGood.get(i);
        }
    }

    System.out.printf("0일 확률 : %.2f%%\n", pG * 100);
    System.out.printf("1일 확률 : %.2f%%\n", pB * 100);
    if (pB > pG) {
        return 1;
    }
    return 0;
}
```

2.2 NBMain.java

2.2.1 문장을 파일에서 읽어서 학습시키는 메소드

```
// 문장을 파일에서 읽어서 학습시키는 메소드
// p : String sentenceFile, String resultFile
private static HashMap<String, Object> trainFromTestFile(String sentenceFile, Stri
    ArrayList<String[]> datas = new ArrayList<>();
    ArrayList<Integer> results = new ArrayList<>();
    FileReader fr = null;       FileReader fr2 = null;
    try {
        fr = new FileReader(sentenceFile);
        BufferedReader br = new BufferedReader(fr);
        String line;
        while ((line = br.readLine()) != null) {
            datas.add(line.split(" "));
        }

        fr2 = new FileReader(resultFile);
        BufferedReader br2 = new BufferedReader(fr2);
        while ((line = br2.readLine()) != null) {
            results.add(Integer.parseInt(line));
        }

        return NaiveBayes.train(datas, results);
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    } finally {
        try {fr.close();fr2.close();} catch (IOException e) {}
    }
}
```

2.2.2 main()

```
public static void main(String[] args) {
    HashMap<String, Object> trainData
        = trainFromTestFile("D:\\Kwon\\자원\\for AI/trainSentences.txt",
        "D:\\Kwon\\자원\\for AI/trainSentencesResult.txt");
    Scanner s = new Scanner(System.in);
    System.out.print("문장 : ");
    String testData = s.nextLine();
    System.out.println(NaiveBayes.getResult(testData, trainData));
    s.close();
}
```

2.3 실행

```
문장 : You are a stupid dog
0일 확률 : 8.33%
1일 확률 : 50.00%
1
```

3. 스팸 메일 필터링

3.1 NBMain.java

3.1.1 메일 읽어서 학습시키는 메소드

```
private static HashMap<String, Object> trainFromMailData(String dir) {
    File dir2 = new File(dir);
    File[] files = dir2.listFiles();
    ArrayList<String[]> datas = new ArrayList<>();
    ArrayList<Integer> results = new ArrayList<>();
    Scanner s = null;
    File f = null;
    BufferedReader br = null;
    String line = null;
    StringBuffer mail;
    String mailTxt = null;
```

```
    try {
        s = new Scanner(System.in);
        for (int i = 0; i < files.length; i++) {
            System.out.printf("%d번 메일\n", i + 1);
            f = files[i];
            System.out.printf("%s\n", f.getPath());
            br = new BufferedReader(new FileReader(f));
            mail = new StringBuffer();
            while ((line = br.readLine()) != null) {
                mail.append(line);
                System.out.println(line);
            }
            mailTxt = mail.toString();
            datas.add(mailTxt.split(" "));
            br.close();
            System.out.print("스팸메일? (1:스팸, 0:아니) : ");
            results.add(s.nextInt());
            System.out.println("-----------------------------------------------
        }
        return NaiveBayes.train(datas, results);
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    } finally {
        s.close();
        try {br.close();} catch (Exception e) {}
    }
}
```

3.1.2 main()

```
public static void main(String[] args) {
    HashMap<String, Object> trainData = trainFromMailData("D:\\Kwon\\자원\\for AI/tr

    StringBuffer mail = new StringBuffer();
    FileReader fr = null;
    try {
        fr = new FileReader("D:\\Kwon\\자원\\for AI/testEMail/mail.txt");
        BufferedReader br = new BufferedReader(fr);
        String line;
        while ((line = br.readLine()) != null) {
            mail.append(line);
        }

        System.out.println(NaiveBayes.getResult(mail.toString(), trainData));
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {fr.close();} catch (IOException e) {}
    }
}
```

3.2 결과

```
50번 메일
D:\Kwon\자원\for AI\trainEMail\9.txt
Hi Peter,

These are the only good scenic ones and it's too ba

D
스팸메일? (1:스팸, 0:아니) : 0
|-----------------------------------------------------
0일 확률 : 9.97%
1일 확률 : 11.11%
1
```